

深圳大学实验报告

课程名称： 计算机系统 1

实验项目名称： 实验五：终端实验

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 李志

报告人： 汉雨 学号： 2024040042 班级： 国际班

实验时间：

实验报告提交时间：

教务处制

一、实验目的

1. 实验目的

使用实验证明：中断驱动的输入、输出可以中断一个正在运行的程序，执行中断服务程序，返回被中断的程序，从被中断位置下一个地址继续执行（好像什么也没发生似的）。试验中我使用键盘作为输入设备，中断正在运行的程序。

二、实验内容

2.1 用户程序

用户程序将会连续地输出纵横交替的 ICS，通过交替，输出两个不同，如下：

ICS ICS ICS ICS ICS ICS

ICS ICS ICS ICS ICS

ICS ICS ICS ICS ICS ICS

ICS ICS ICS ICS ICS

ICS ICS ICS ICS ICS ICS

ICS ICS ICS ICS ICS

ICS ICS ICS ICS ICS ICS

ICS ICS ICS ICS ICS

确保输出不至于太快，以至于肉眼不能察觉，因而，需要延时操作，可以考虑如下实现：

即用户程序包含一小段代码用于每行间的计数，间隔为从 2500 开始倒计时，计时结束时，再进行输出。

2.2 键盘中断服务程序

键盘中断服务程序将会简单地在屏幕上写上十次用户随机输入的字符并以 Enter (x0A) 结束。

中断服务程序中要求不使用 TRAP 指令。

注意，不要忘记保存和恢复在中断服务程序中使用的寄存器。

2.3 模拟操作系统支持

不幸的是，我们还没在 LC-3 上安装 Windows 或 Linux，所以我们必须要求你在你的用户程序代码前先做到以下三个步骤：

1. 正常情况下，操作系统将会先安装一些特定词，所以当中断发生的时候，PC 和 PSR 可以被放进栈中（正如你知道的，当程序执行 RTI，PC 和

- PSR 都会被弹出栈，处理器返回到执行被中断的程序中；于没有操作系统，请先把 R6 初始化为 x4000，表示一个空栈）。
2. 正常情况下，操作系统会建立中断向量表，它包含对应中断服务程序的起始地址，你必须为键盘中断先建立一个中断向量表。中断向量表的开始地址是 x0100，键盘中断的中断向量是 x80。你必须在中断向量表提供一个入口供本实验使用。
- 操作系统应该设置 KBSR 的 IE（Interrupt Enable）位。

三、实验步骤与结果

本部分围绕 main_code2.txt 与 sub_code2.txt 展开。主程序负责建立中断环境并持续输出 ICS；中断服务程序负责接收键盘输入，并在不使用 TRAP 的前提下将输入内容连续输出 10 次。两部分配合后，可以直观看到 主程序被键盘输入打断、ISR 执行结束后又回到原位置继续运行 的完整过程。

表 1 中断实验关键机制概览

项目	说明
中断向量表项	$x0100 + x80 = x0180$ ，主程序把 ISR 入口地址写入该位置。
中断服务入口	x2000，即 sub_code2.txt 的 .ORIG 起始地址。
中断允许位	KBSR[14] 置 1 后，键盘输入才能打断正在运行的主程序。
现场保护	硬件自动保护 PC/PSR，ISR 手动保存 R0-R4，最后用 RTI 返回。

3.1 主程序 main_code2.txt 讲解

主程序首先完成“模拟操作系统”的初始化工作：用 R6 指向栈空间，为后续中断现场保护提供栈；随后把键盘中断服务程序入口地址 x2000 写入键盘中断向量表项 x0180。完成向量表设置后，程序把 KBSR[14] 置 1，允许键盘产生中断。

初始化结束后，主程序进入实际用户任务：循环输出两行略微错位的“ICS”字符串，并在两次输出之间调用 DELAY 子程序。这个延时不是中断机制必需的，但能让控制台输出速度降低，便于观察输入触发中断前后的变化。由于主循环通过 BRnzp AGAIN 永远回到开头，所以键盘输入可以在任意时刻打断正在输出的主程序。

主程序重点

中断向量表：把 x2000 写入 M[x0180]，让键盘中断能找到 ISR。

中断允许位：KBSR[14] 置 1 后，键盘输入才会真正打断主程序。

循环输出：STR1 与 STR2 交替输出，构成实验中观察到的 ICS 棋盘效果。

表 2 main_code2.txt 主程序核心代码

行	代码
1	.ORIG x3000
2	LD R6, STACK ; 关键：初始化系统栈指针，为中断现场保存做准备
3	LD R1, ENTRY1 ; R1 <- x0180, 定位键盘中断向量表项
4	LD R2, STARTS ; R2 <- x2000, 中断服务程序入口地址
5	STR R2, R1, #0 ; 关键：M[x0180] <- x2000，建立键盘中断入口
6	LD R3, IE ; x4000 只置位 KBSR[14]
7	STI R3, KBSR ; 关键：打开键盘中断允许位 IE
9	AGAIN LEA R0, STR1 ; 输出第一行 ICS 棋盘图案
10	PUTS
11	JSR DELAY ; 延时，保证肉眼能观察到交替输出
12	LEA R0, STR2 ; 输出第二行，形成错位交替效果
13	PUTS
14	JSR DELAY
15	BRnzp AGAIN ; 主程序持续运行，等待键盘中断随时打断
17	ENTRY1 .FILL x0180 ; 键盘中断向量表项地址
18	STARTS .FILL x2000 ; ISR 起始地址
19	STACK .FILL x3000 ; 栈顶地址
20	IE .FILL x4000 ; KBSR[14] = 1

21	KBSR .FILL xFE00 ; 键盘状态寄存器
22	STR1 .STRINGZ "ICS ICS ICS ICS ICS ICS \n"
23	STR2 .STRINGZ " ICS ICS ICS ICS ICS ICS \n"
25	DELAY ST R1, SaveR1 ; 保存延时子程序使用的 R1
26	LD R1, COUNT
27	REP ADD R1, R1, #-1
28	BRp REP
29	LD R1, SaveR1
30	RET
31	COUNT .FILL #25000 ; 延时计数值, 可按运行速度调整
32	SaveR1 .BLKW 1
33	.END

主程序流程图：

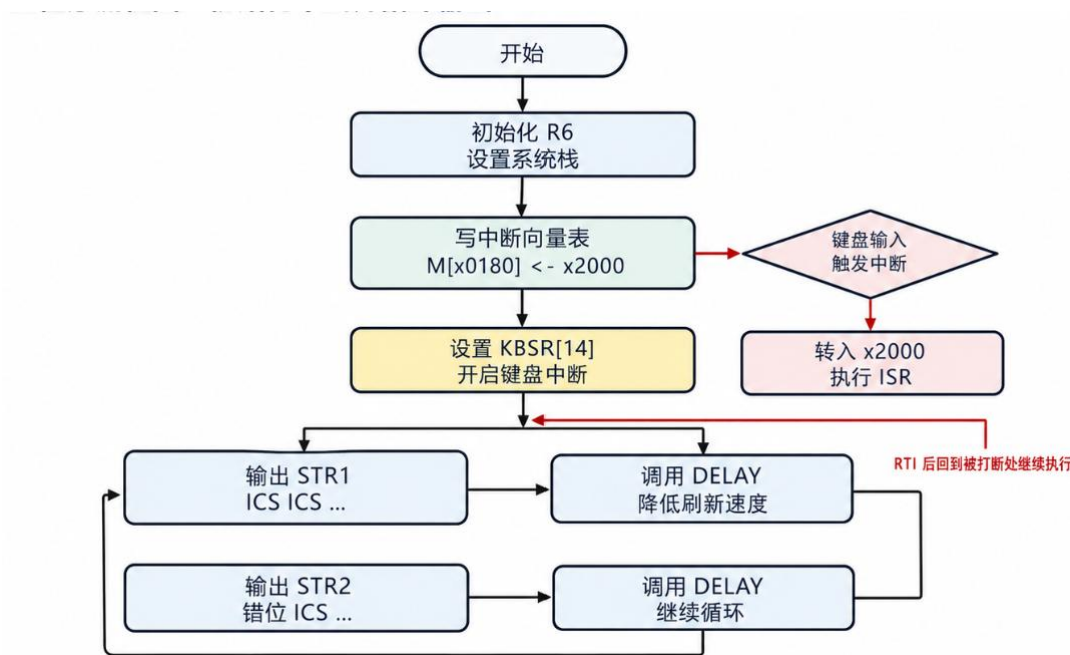


图 1 主程序流程图：初始化中断后持续输出 ICS

3.2 中断服务程序 sub_code2.txt 讲解

键盘中断发生后，LC-3 硬件会自动把 PC 与 PSR 压入栈中，但中断服务程序内部使用到的通用寄存器仍需手动保存。因此本程序开头依次保存 R0-R4，

结束时再按照相反顺序恢复，保证返回主程序后不会破坏原来的运行状态。

输入阶段，程序通过 **KBSR** 判断键盘是否准备好，再从 **KBDR** 读取字符；若读到 **Enter**，即 **x000A**，则停止接收并在缓冲区末尾写入 **x0000**。输出阶段，程序把计数器 **R3** 设为 **10**，每一轮从 **x4000** 重新遍历字符串，等待 **DSR** 就绪后直接写入 **DDR** 输出字符。这里 **不使用 TRAP 指令**，而是直接访问设备寄存器，符合中断服务程序的实验要求。

- 保存现场：**R0-R4** 入栈，避免 **ISR** 改写主程序正在使用的寄存器。
- 接收字符串：普通字符依次写入 **x4000** 开始的缓冲区，**Enter** 表示输入结束。
- 重复输出：每次从字符串首地址开始输出，直到计数器完成 **10** 轮。
- **返回主程序：恢复寄存器并执行 RTI，主程序继续输出 ICS。**

表 3 sub_code2.txt 中断服务程序核心代码

行	代码（核心节选，已补充中文注释）
1	.ORIG x2000
2	ADD R6, R6, #-1 ; 关键：压栈保存 R0
3	STR R0, R6, #0
4	ADD R6, R6, #-1 ; 保存 R1
5	STR R1, R6, #0
6	ADD R6, R6, #-1 ; 保存 R2
7	STR R2, R6, #0
8	ADD R6, R6, #-1 ; 保存 R3
9	STR R3, R6, #0
10	ADD R6, R6, #-1 ; 保存 R4
11	STR R4, R6, #0
13	LD R4, STRING ; R4 指向输入缓冲区 x4000
14	READ_LOOP
15	LDI R0, KBSR ; 读取键盘状态寄存器

16	BRzp READ_LOOP ; 未准备好则等待
17	LDI R0, KBDR ; 读取一个键盘字符
18	LD R2, NEG_ENTER
19	ADD R2, R0, R2
20	BRz INPUT_END ; 读到 Enter(x000A) 后结束输入
21	STR R0, R4, #0 ; 保存普通字符到缓冲区
22	ADD R4, R4, #1
23	BRnzp READ_LOOP
25	INPUT_END
26	AND R0, R0, #0
27	STR R0, R4, #0 ; 关键: 补 x0000, 形成 LC-3 字符串
28	AND R3, R3, #0
29	ADD R3, R3, #10 ; 设置重复输出次数为 10
31	PRINT_10
32	LD R4, STRING ; 每轮从字符串首地址重新开始
33	PRINT_CHAR
34	LDR R0, R4, #0
35	BRz ONE_TIME_DONE ; 字符串结束, 一轮输出完成
36	WAIT_DSR
37	LDI R1, DSR ; 查询显示器状态寄存器
38	BRzp WAIT_DSR ; 显示器未就绪则等待
39	STI R0, DDR ; 关键: 不用 TRAP, 直接写 DDR 输出字符
40	ADD R4, R4, #1
41	BRnzp PRINT_CHAR

43	ONE_TIME_DONE
44	ADD R3, R3, #-1
45	BRp PRINT_10 ; 未滿 10 次則繼續輸出
47	LDR R4, R6, #0 ; 按相反順序恢復寄存器
48	ADD R6, R6, #1
49	LDR R3, R6, #0
50	ADD R6, R6, #1
51	LDR R2, R6, #0
52	ADD R6, R6, #1
53	LDR R1, R6, #0
54	ADD R6, R6, #1
55	LDR R0, R6, #0
56	ADD R6, R6, #1
57	RTI ; 关键: 从中断返回, 主程序继续输出 ICS
59	KBSR .FILL xFE00
60	KBDR .FILL xFE02
61	DSR .FILL xFE04
62	DDR .FILL xFE06
63	NEG_ENTER .FILL xFFF6 ; -x000A
64	STRING .FILL x4000 ; 輸入緩沖區首地址
65	.END

中断服务程序流程图：读取输入并重复输出 10 次

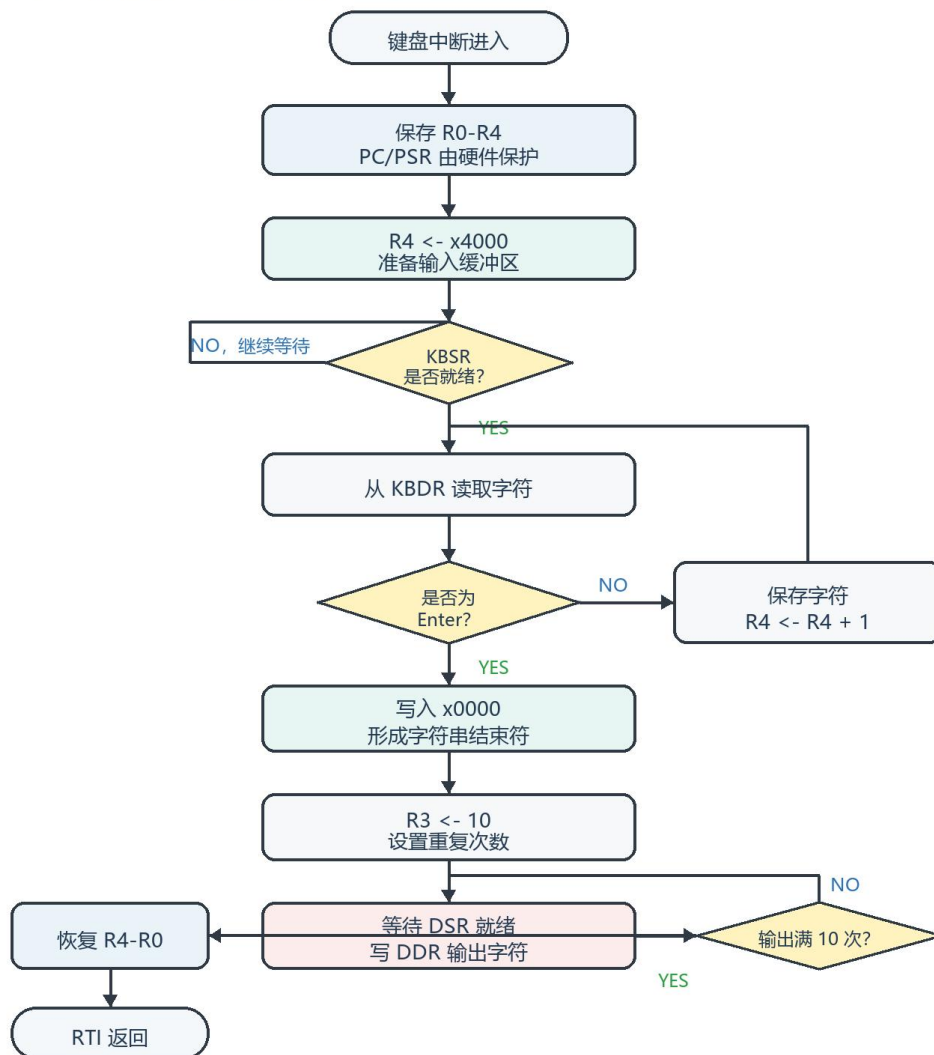


图 2 中断服务程序流程图：读取输入并连续输出 10 次

3.3 实验运行步骤与结果分析

实验时先分别汇编并加载主程序与中断服务程序，再从主程序入口 `x3000` 开始运行。主程序启动后会不断输出纵横交替的 ICS；当控制台接收到键盘输入并按下 Enter 后，处理器转入中断服务程序，完成输入字符串的 10 次输出，再返回原主程序继续运行。

(1) 主程序初始循环输出 “ICS”

未输入任何字符时，程序持续执行主循环。两条字符串 STR1 与 STR2 前导空格不同，因此控制台中出现交替错位的 “ICS” 图案，说明主程序已经稳定运行，并且延时子程序使输出节奏可以被观察到。

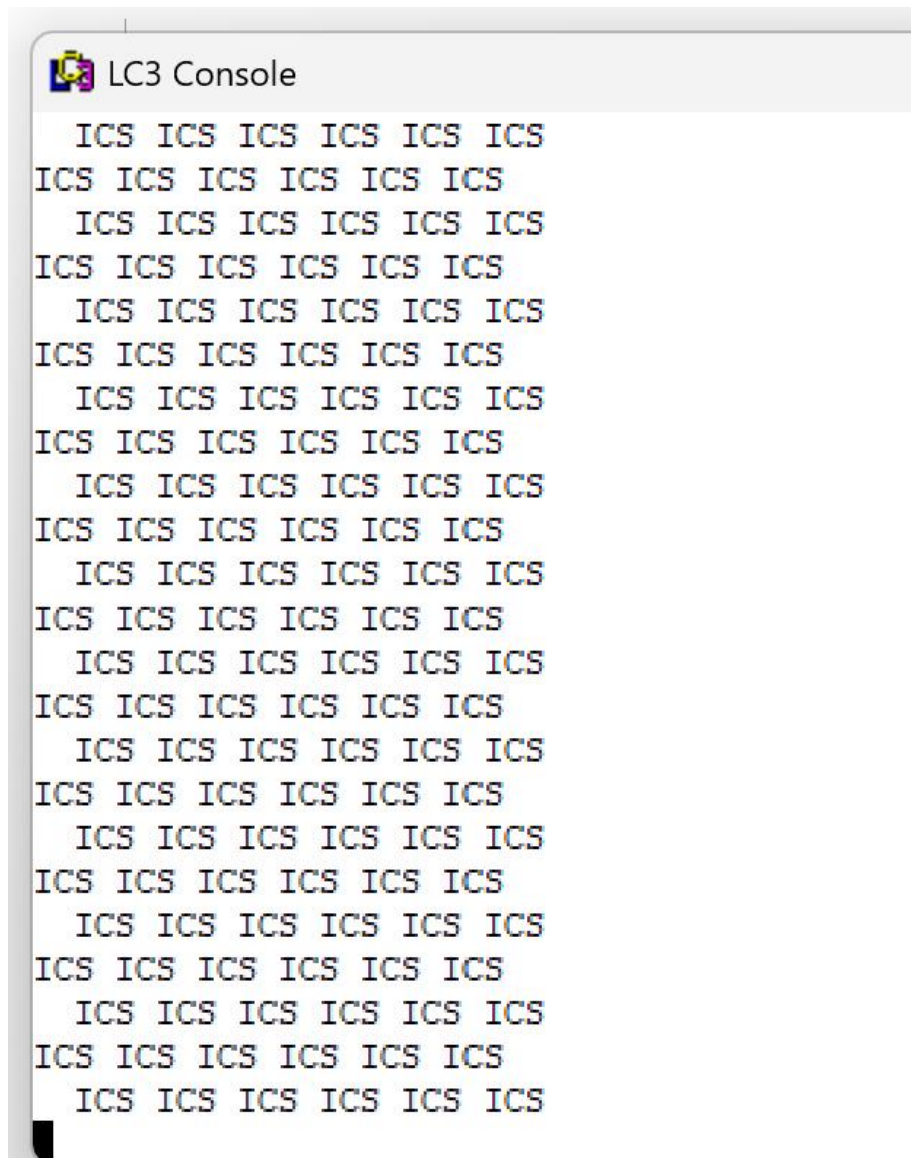
The image shows a screenshot of a terminal window titled "LC3 Console". The window contains a repeating pattern of the text "ICS" followed by a space, repeated six times per line. This pattern is repeated for 20 lines in total, creating a dense grid of the text "ICS ICS ICS ICS ICS ICS". The text is in a monospaced font, and the background is white with a light gray border around the terminal area.

图 3 主程序初始运行结果：循环交替输出 “ICS”

(2) 输入 “a” 触发中断

当主程序正在输出 ICS 时输入字符 “a” 并按 Enter，键盘中断立即打断主循环。中断服务程序读取到单字符字符串后，将其连续输出 10 次，随后通过 RTI 返回主程序。因此图中可以看到一串 **aaaaaaaaaa** 插入在 ICS 输出流中，证明中断服务程序确实在主程序执行过程中被调用。

```
LC3 Console
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
aaaaaaaaaaaaICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
```

图 4 输入“a”后的中断输出结果：连续输出 10 次 a

(3) 输入“abc”触发中断

继续测试多字符输入。输入“abc”后，ISR 先把 a、b、c 依次存入 x4000 起始的缓冲区，并在 Enter 处补上字符串结束符；输出阶段每一轮都从缓冲区首地址重新开始，因此最终显示为 **abcabcabcabcabcabcabcabcabcabc**。这说明程序不仅能处理中断，还能正确保存并重复输出完整字符串。

```
LC3 Console
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
abcabcabcabcabcabcabcabcabcabc ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
ICS ICS ICS ICS ICS ICS
```

图 5 输入“abc”后的中断输出结果：字符串 abc 连续输出 10 次

四、实验结论

本次实验完成了 LC-3 键盘中断的完整验证。主程序通过设置栈指针、填写键盘中断向量表项并打开 KBSR 的 IE 位，建立了中断发生所需的基本环境；随后在循环输出 ICS 的过程中等待键盘输入。中断服务程序在 x2000 处启动，

先保存现场，再读取用户输入并将字符串连续输出 10 次，最后恢复寄存器并执行 RTI 返回主程序。

从实验现象看，无论输入单个字符“a”还是字符串“abc”，输出都能插入到主程序的 ICS 输出流中，并在中断处理完成后继续回到主循环。这说明 中断不是替代主程序执行，而是在必要时临时接管处理器，完成任务后再把控制权交回原程序。同时，实验也加深了我对中断向量表、设备寄存器、现场保护与 RTI 返回机制的理解。

实验结论要点

1. 键盘中断入口由 M[x0180] 指向 x2000，向量表设置正确是 ISR 能执行的前提。
2. ISR 必须保存并恢复使用过的通用寄存器，否则会破坏主程序状态。
3. 直接访问 KBSR/KBDR/DSR/DDR 可以在不使用 TRAP 的情况下完成输入输出。
4. RTI 返回后，主程序能够从被中断的位置继续执行，实验目标得到验证。

指导教师批阅意见:

成绩评定：

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。